

Multi-Query and Grouped-Query Attention – Sharing K/V

A structural and numerical demonstration of sharing K/V heads.

Notation and abbreviations

Symbol / abbreviation	Meaning
MHA	Multi-Head Attention
GQA	Grouped-Query Attention
MQA	Multi-Query Attention
Q, K, V	query, key, and value representations
H_q	number of query heads
H_{kv}	number of key/value heads
d_{model}	model hidden-state width
d_k	width of each query/key head
d_v	width of each value head
G	query heads per K/V group, $G = H_q/H_{kv}$
$g(h)$	K/V-head index assigned to query head h
W_Q, W_K, W_V	query, key, and value projection matrices
W_O	output projection matrix
B	batch size
L	cached sequence length
N_{layers}	number of Transformer layers

1. Define H_q and H_{kv} .

Let H_q be the number of query heads in the attention layer, and H_{kv} be the number of distinct key/value head pairs. A primary inference benefit of decoupling H_{kv} from H_q is reduced KV-cache memory and decoding-time memory bandwidth.

2. For MHA with $H_q = 4$, determine H_{kv} .

In Multi-Head Attention (MHA), every query head has its own K/V head. Therefore, $H_{kv} = H_q = 4$.

3. For GQA with $H_q = 4$ and $H_{kv} = 2$, compute the group size G .

In Grouped-Query Attention (GQA), the query heads are partitioned into H_{kv} equal-sized groups. The number of query heads per group is $G = H_q/H_{kv} = 4/2 = 2$.

4. Map each query head to its K/V head for GQA.

Using the 1-based mathematical mapping $g(h) = \lfloor (h-1)/G \rfloor + 1$, the query heads ($h \in \{1, 2, 3, 4\}$) share the $H_{kv} = 2$ K/V heads as follows:

$$\begin{aligned} g(1) &= \lfloor 0/2 \rfloor + 1 = 1 \implies Q_1 \rightarrow K_1, V_1 \\ g(2) &= \lfloor 1/2 \rfloor + 1 = 1 \implies Q_2 \rightarrow K_1, V_1 \\ g(3) &= \lfloor 2/2 \rfloor + 1 = 2 \implies Q_3 \rightarrow K_2, V_2 \\ g(4) &= \lfloor 3/2 \rfloor + 1 = 2 \implies Q_4 \rightarrow K_2, V_2 \end{aligned}$$

With 0-based indexing in code, the same mapping is simply $g(h) = \lfloor h/G \rfloor$, or `h // G` in Python.

5. For MQA, map all query heads to the single K/V head.

In Multi-Query Attention (MQA), $H_{kv} = 1$, yielding a single group of size $G = 4/1 = 4$. Consequently, $g(h) = 1$ for all 1-based head indices h .

$$Q_1, Q_2, Q_3, Q_4 \rightarrow K_1, V_1$$

6. Derive the general head formula using $g(h)$.

The attention operation for any query head h uses its assigned K/V group $g(h)$. The output of head h is a weighted sum of the rows of $V_{g(h)}$ under the attention distribution:

$$\text{head}_h = \text{softmax} \left(\frac{Q_h K_{g(h)}^\top}{\sqrt{d_k}} \right) V_{g(h)}$$

7. Use the supplied numerical example for two query heads sharing K/V.

To show that sharing K/V heads does not force query heads to become identical, evaluate two distinct queries sharing the same K/V state:

$$\begin{aligned} q_1 &= \begin{bmatrix} 1 & 0 \end{bmatrix}, & q_2 &= \begin{bmatrix} 0 & 1 \end{bmatrix} \\ K_{\text{shared}} &= \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, & V_{\text{shared}} &= \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \end{aligned}$$

8. Compute both raw score vectors $q_1 K_{\text{shared}}^\top$ and $q_2 K_{\text{shared}}^\top$.

The different queries produce distinct raw logits against the shared keys:

$$\begin{aligned} S_1 &= q_1 K_{\text{shared}}^\top = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \\ &= [(1)(1) + (0)(0) \quad (1)(0) + (0)(1)] = \begin{bmatrix} 1 & 0 \end{bmatrix} \\ S_2 &= q_2 K_{\text{shared}}^\top = \begin{bmatrix} 0 & 1 \\ 0 & 1 \end{bmatrix} \\ &= [(0)(1) + (1)(0) \quad (0)(0) + (1)(1)] = \begin{bmatrix} 0 & 1 \end{bmatrix} \end{aligned}$$

9. Scale both by $\sqrt{d_k}$.

Assuming q and k have zero-mean, unit-variance components, their dot product has variance proportional to d_k . Dividing by $\sqrt{d_k}$ keeps logits from growing with head width. Here, $\sqrt{d_k} = \sqrt{2} \approx 1.4142$:

$$\hat{S}_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 0 \end{bmatrix} \approx \begin{bmatrix} 0.7071 & 0 \end{bmatrix}, \quad \hat{S}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 0 & 1 \end{bmatrix} \approx \begin{bmatrix} 0 & 0.7071 \end{bmatrix}$$

10. Compute both softmax distributions.

The softmax function exponentiates and normalizes the logits:

$$\begin{aligned} Z_1 &= e^{0.7071} + e^0 \approx 2.0281 + 1.0000 = 3.0281 \\ A_1 &= \frac{1}{Z_1} \begin{bmatrix} e^{0.7071} & e^0 \end{bmatrix} \approx \begin{bmatrix} 0.6698 & 0.3302 \end{bmatrix} \\ Z_2 &= e^0 + e^{0.7071} \approx 1.0000 + 2.0281 = 3.0281 \\ A_2 &= \frac{1}{Z_2} \begin{bmatrix} e^0 & e^{0.7071} \end{bmatrix} \approx \begin{bmatrix} 0.3302 & 0.6698 \end{bmatrix} \end{aligned}$$

11. Compute both outputs.

Each output is a convex combination (weighted sum) of the rows of V_{shared} , using the corresponding attention probabilities:

$$\begin{aligned} O_1 &= A_1 V_{\text{shared}} = \begin{bmatrix} 0.6698 & 0.3302 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \\ &= [(0.6698)(1) + (0.3302)(1) \quad (0.6698)(0) + (0.3302)(1)] = \begin{bmatrix} 1.0000 & 0.3302 \end{bmatrix} \\ O_2 &= A_2 V_{\text{shared}} = \begin{bmatrix} 0.3302 & 0.6698 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \\ &= [(0.3302)(1) + (0.6698)(1) \quad (0.3302)(0) + (0.6698)(1)] = \begin{bmatrix} 1.0000 & 0.6698 \end{bmatrix} \end{aligned}$$

12. Verify that shared K/V still gives different attention distributions and outputs.

Since $A_1 \neq A_2$ and $O_1 \neq O_2$, query heads that share K/V can still produce distinct attention distributions and distinct outputs.

13. Explain why fewer KV heads does not mean fewer query heads.

The number of query heads is still H_q . Even if $H_{kv} = 1$, the model produces H_q distinct query vectors, computes H_q distinct attention distributions, and produces H_q per-head outputs that are concatenated before the final W_O projection.

14. Compute Q , K , V , and output projection widths for MHA, GQA, and MQA.

Given $H_q = 4$, $d_k = d_v = 2$, and $d_{\text{model}} = 8$:

- **MHA** ($H_{kv} = 4$): Q width $= 4 \times 2 = 8$, K width $= 4 \times 2 = 8$, V width $= 4 \times 2 = 8$.
- **GQA** ($H_{kv} = 2$): Q width $= 4 \times 2 = 8$, K width $= 2 \times 2 = 4$, V width $= 2 \times 2 = 4$.
- **MQA** ($H_{kv} = 1$): Q width $= 4 \times 2 = 8$, K width $= 1 \times 2 = 2$, V width $= 1 \times 2 = 2$.

For the query projection, the first matrix dimension is the input width d_{model} , and the second is the total query-projection output width $H_q d_k$:

$$W_Q : \underbrace{8}_{d_{\text{model}}} \times \underbrace{(4 \cdot 2)}_{H_q d_k} = 8 \times 8.$$

Therefore,

$$W_Q \in \mathbb{R}^{8 \times 8},$$

while

$$W_K, W_V : 8 \times 8 \rightarrow 8 \times 4 \rightarrow 8 \times 2.$$

After concatenating the $H_q = 4$ head outputs, the width is

$$H_q d_v = 4 \times 2 = 8,$$

so

$$W_O \in \mathbb{R}^{(H_q d_v) \times d_{\text{model}}} = \mathbb{R}^{8 \times 8}$$

for MHA, GQA, and MQA. Therefore W_Q and W_O stay the same size, while W_K and W_V shrink with H_{kv} .

15. Compute combined K/V projection parameter counts.

Assuming bias-free linear projections, the combined number of *weight* parameters in W_K and W_V is

$$d_{\text{model}} H_{kv} (d_k + d_v).$$

With $d_{\text{model}} = 8$ and $d_k = d_v = 2$:

- **MHA**: $8 \times 4 \times 4 = 128$ weights.
- **GQA**: $8 \times 2 \times 4 = 64$ weights.
- **MQA**: $8 \times 1 \times 4 = 32$ weights.

These counts refer only to W_K and W_V . In this toy setup, W_Q and W_O remain 8×8 across all three architectures.

16. Write K_{cache} and V_{cache} tensor shapes.

For one Transformer layer during decoding,

$$K_{\text{cache}} \in \mathbb{R}^{B \times H_{kv} \times L \times d_k}, \quad V_{\text{cache}} \in \mathbb{R}^{B \times H_{kv} \times L \times d_v},$$

where B is batch size and L is cached sequence length.

17. Derive the general cache element formula.

For one layer, the number of cached scalar elements is

$$BLH_{kv}d_k + BLH_{kv}d_v = BLH_{kv}(d_k + d_v).$$

Across N_{layers} Transformer layers,

$$\boxed{BN_{\text{layers}}LH_{kv}(d_k + d_v)}.$$

This is an element count. The actual memory footprint in bytes is

$$\boxed{\text{KV-cache bytes} = BN_{\text{layers}}LH_{kv}(d_k + d_v) \times \text{bytes per element}}.$$

For FP16 or BF16, bytes per element = 2.

18. Compute cache elements for MHA, GQA, and MQA with $L = 3$.

For **one layer** with $B = 1$, $L = 3$, and $d_k = d_v = 2$, the cache formula simplifies to $4LH_{kv}$:

- **MHA** ($H_{kv} = 4$): $4 \times 3 \times 4 = 48$ scalars.
- **GQA** ($H_{kv} = 2$): $4 \times 3 \times 2 = 24$ scalars.
- **MQA** ($H_{kv} = 1$): $4 \times 3 \times 1 = 12$ scalars.

For FP16/BF16, **under these same assumptions** ($B = 1$, $L = 3$, **one layer**), these correspond to 96, 48, and 24 bytes respectively.

19. Explain what gets cheaper as H_{kv} decreases.

Reducing H_{kv} decreases the K/V projection parameter count, KV-cache memory, and the decoding-time memory bandwidth required to read cached keys and values. K/V projection arithmetic also decreases, but the main serving benefit is the smaller cache and lower decode-time memory traffic.

20. Explain what does NOT disappear when H_{kv} decreases.

The number of query heads H_q remains unchanged. The model still performs H_q distinct query-head attention lookups per token, and the final output width remains unchanged.

21. Explain the MHA $\rightarrow$ GQA $\rightarrow$ MQA structural continuum.

MHA uses a separate K/V head for each query head, MQA uses one K/V head shared by all query heads, and GQA lies between them by sharing each K/V head across a group of query heads.

Final conceptual question

How can MQA use only one K/V head while still retaining multiple distinct query-head attention patterns?

The query projection remains

$$W_Q \in \mathbb{R}^{d_{\text{model}} \times (H_q d_k)}.$$

Its output is interpreted as H_q distinct query heads. Those distinct query vectors all attend to the same shared K/V head, but because their Q_h vectors differ, they produce different $Q_h K^\top$ scores, different softmax distributions, and potentially different outputs.

Optional bonus question

If sequence length L doubles, what happens to the cache size for MHA, GQA, and MQA? Do their relative ratios change?

Because the cache element count is linear in L , doubling L doubles the KV-cache element count and byte footprint for all three architectures. In this toy setup, the relative ratio remains $4 : 2 : 1$ because H_{kv} is 4, 2, 1 respectively. In general, the relative cache ratio is determined by the corresponding H_{kv} values.

Pipeline Summary

MHA: H_q query heads $\rightarrow H_q$ K/V heads $\rightarrow$ each Q head uses its own K/V

GQA: H_q query heads $\rightarrow H_{kv}$ K/V heads $\rightarrow$ groups of Q heads share K/V

MQA: H_q query heads $\rightarrow 1$ K/V head $\rightarrow$ all Q heads share K/V

Numerical Verification

A deterministic PyTorch verifier (`answer_key.py`) verifies the head mappings, shared-K/V attention arithmetic, distinct outputs, and the parameter/cache ratios used in this exercise.